

Clustering and Initialization

1 Introduction

This work is based on the clustering problem, which involves grouping data points considered similar into clusters. The desired k clusters and a set of n data points in a d -dimensional space are provided. This problem is also formulated as the minimisation of the cost function, defined as the sum of the squared distances between each point and its nearest cluster centre. The difficulty lies in the fact that this problem is considered NP-hard; therefore, it is computationally challenging to minimise the function in polynomial time. However, two algorithms have been introduced: K-means and K-means++. The first algorithm alternates between assigning points to each cluster and updating the choice of new centres for each cluster, but it is vulnerable due to its random initialisation of centres, which leads to the problem of getting stuck in a local optimum of the cost function. The second algorithm, on the other hand, solves the problem by selecting new centres based on a probability weighted by the square of the distance from the centres already chosen. It guarantees that the expected value of the cost function is at most $8(\ln k + 2)$ times worse than the optimal cost function, providing a logarithmic approximation guarantee that standard K-means cannot achieve.

2 Data

2.1 Synthetic dataset

A synthetic dataset was used with a 12-dimensional space and a space-side length of 400 for the hypercube, in which 20 initial centres were generated according to a uniform distribution over the interval $[-200, 200]$ for all 12 dimensions. For each of these centres, 500 data points were generated from a normal distribution with a small spread of points to maximise the potential for good clustering, with a standard deviation of 1.

2.2 Reral dataset

When selecting a real-world dataset from those available on the UCI Machine Learning Repository website, the ‘Human Activity Recognition Using Smartphones’ dataset proved to be the ideal choice for testing the algorithms. This dataset was created by recording 30 participants carrying out everyday activities whilst wearing a smartphone fitted with inertial sensors on their hip. Each person was asked to perform six activities (WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING, LAYING), and using the smartphone’s accelerometers and gyroscope, the three linear axes of acceleration and three angular axes of velocity were captured at a constant sampling rate of 50 Hz. The dataset is randomly split into two subsets: 70% for training and 30% for testing. However, as clustering is an unsupervised learning problem and the aim is to explore the individual data points across the algorithms rather than to make predictions, there is no

need to use labels; instead, we simply need to calculate and minimise the cost function. Therefore, the two sets have been merged. Furthermore, each of the 561 features in the time and frequency domains has been pre-processed using normalisation and restricted to the interval $[-1, 1]$. Finally, due to the similarity in the dynamic and static movements, respectively, across the six activities, the clusters may potentially overlap with one another, and we could compare this dataset with the synthetic one, for which, by contrast, a good potential for clustering was required.

3 Algorithms

3.1 K-means standard

The first algorithm used was the one proposed by Stuart P. Lloyd, namely the standard k-means algorithm, which consists of the following steps:

1. Choose k initial centres arbitrarily (typically by selecting k random points from the dataset).
2. Assign each point in the dataset to the nearest centre, thereby forming k clusters.
3. Recalculate each centre as the centre of mass (the arithmetic mean) of all the points assigned to that cluster.
4. Repeat steps 2 and 3 until the centres stop moving.

It has been shown, however, that the latter is vulnerable. The algorithm is highly sensitive to its initialisation (Step 1). If the initial centres are chosen poorly, the algorithm is likely to converge to a sub-optimal local minimum of the cost function.

3.2 K-means++

David Arthur and Sergei Vassilvitskii have therefore devised an excellent solution, known as k-means++, to mitigate this initialisation issue. The central idea is that the initial cluster centres should be chosen as far apart from one another as possible. Thus, k-means++ replaces only the initialisation phase (Step 1) of standard k-means with the following probabilistic sampling approach.

1. Select the first centre c_1 by choosing it uniformly at random from the data set X .
2. Choose the next cluster centre $c_i = x'$ with a probability proportional to its contribution to the current potential. The exact formula for the probability is:

$$P(x') = \frac{D(x')^2}{\sum_{x \in X} D(x)^2}$$

3. Repeat Step 2 until all k cluster centres have been selected.
4. Proceed with Steps 2–4 of the standard k-means algorithm to refine the positions of the clusters

3.3 Implementations

```
def ricercaCentriFinali(X, k, c, features):
    while True:
        differenze = X[:, np.newaxis, :] - c[np.newaxis, :, :]
        distanze = np.sqrt(np.sum(differenze ** 2, axis=2))
        labels = np.argmin(distanze, axis=1)

        nuovi_centri = np.zeros((k, features))

        for i in range(k):
            punti_nel_cluster = X[labels == i]

            if len(punti_nel_cluster) > 0:
                nuovi_centri[i] = np.mean(punti_nel_cluster, axis=0)
            else:
                nuovi_centri[i] = c[i]

        if np.allclose(c, nuovi_centri):
            break

        c = nuovi_centri

    return c, labels

def kmeans_standard(X, k):
    n_campioni, n_feature = X.shape

    indici_casuali = np.random.choice(n_campioni, size=k, replace=False)
    centri = X[indici_casuali].copy()

    centri, labels = ricercaCentriFinali(X, k, centri, n_feature)

    return centri, labels
```

Listing 1: Implementation of K-Means standard

For both algorithms, the method of searching for final centres was used, which calculates all the distances from each point to each centre and moves the centres of each cluster until they find their perfect barycenter. In standard K-means, we note that the first k clusters are chosen at random, and this is where the algorithm's weakness lies, with the risk of converging to a local minimum.

```

def kmeans_plpl(X, k):
    n_campioni, n_feature = X.shape
    centri = np.zeros((k, n_feature))

    primo_indice = np.random.choice(n_campioni)
    centri[0] = X[primo_indice]

    for i in range(1, k):

        differenze = X[:, np.newaxis, :] - centri[:i][np.newaxis, :, :]
        distanze_quadrate = np.sum(differenze ** 2, axis=2)

        D2 = np.min(distanze_quadrate, axis=1)

        somma_D2 = np.sum(D2)
        if somma_D2 == 0:
            prob = np.ones(n_campioni) / n_campioni
        else:
            prob = D2 / somma_D2

        nuovo_indice = np.random.choice(n_campioni, p=prob)
        centri[i] = X[nuovo_indice]

    centri, labels = ricercaCentriFinali(X, k, centri, n_feature)

    return centri, labels

```

Listing 2: Implementation of the K-Means++ algorithm

Here, it replaces the random approach of selecting the first k centres; one is chosen at random, and the remaining $k - 1$ centres are then selected in a more controlled and weighted manner using the minimum squared distance $D(x)^2$, so as to position the $k - 1$ centres as far away as possible from those already chosen.

4 Experiments

To analyse the differences between the two algorithms, various values of k (number of clusters) were tested across a total of 20 independent trials, on both the synthetic and real datasets. The aim was therefore to demonstrate how the algorithms behave differently when the complexity of the clustering is varied in relation to the actual nature of the datasets. To this end, graphs were generated to visualise the distributions of the data points assigned for each value of k , accompanied by box plots comparing the stability of the inertia (cost function) of the two algorithms. The different distributions observed are as follows:

4.1 Standard K-means on a synthetic dataset

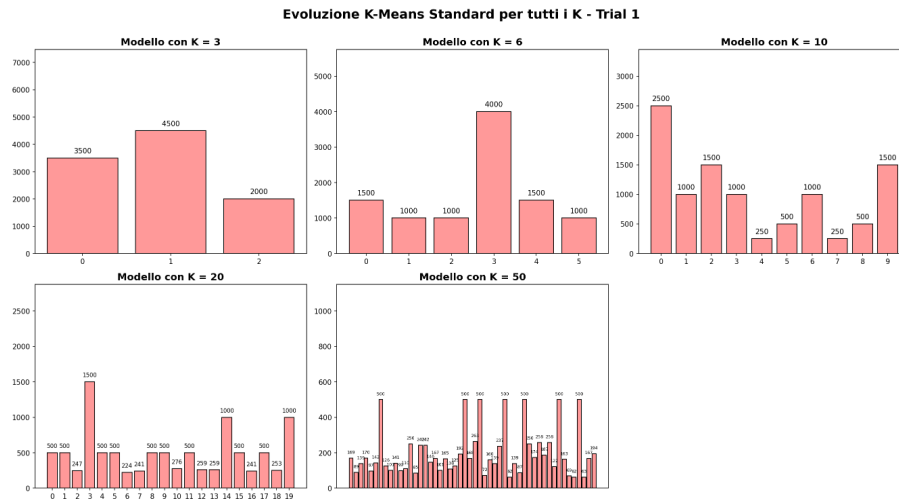


Figure 1: Distributions on a synthetic dataset using standard K-means. [To view other trials.](#)

4.2 K-means++ on a synthetic dataset

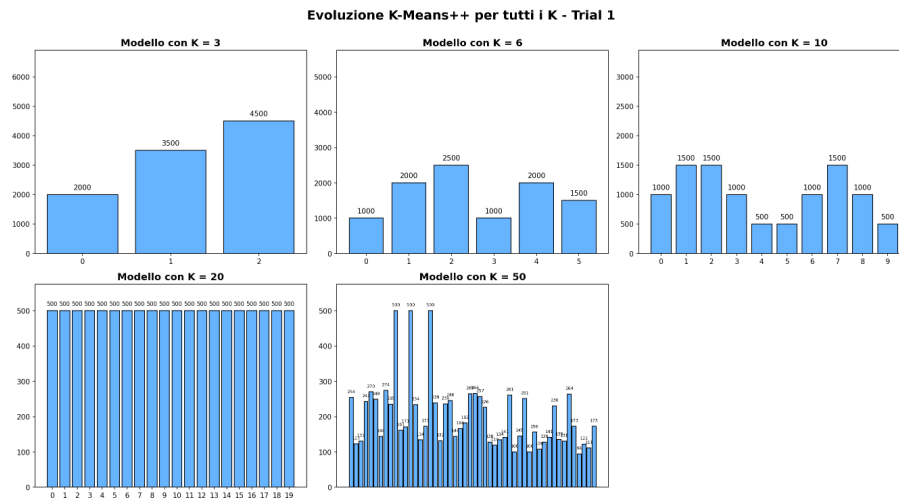


Figure 2: Distributions on a synthetic dataset using K-means++. [To view other trials.](#)

Here we can see that, for $k = 20$, the standard K-means algorithm struggled more to distribute the data points evenly, unlike K-means++, on a dataset that was created with clearly distinct clusters.

4.3 Standard K-means on the HAR dataset

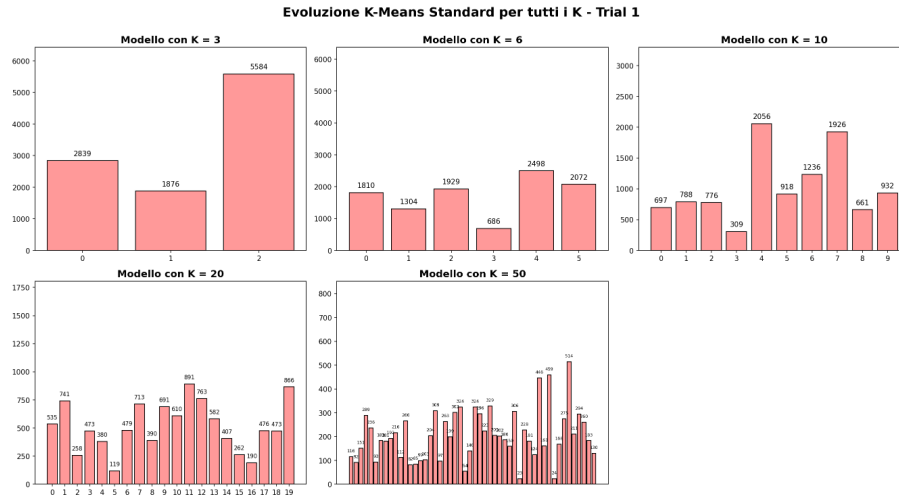


Figure 3: Distributions on HAR datasets using standard K-means. [To view other trials.](#)

4.4 K-means++ on the HAR dataset

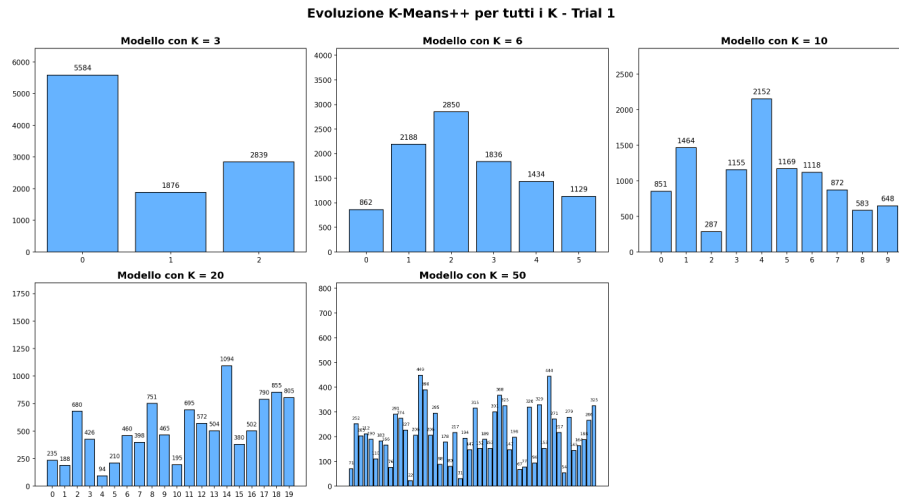


Figure 4: Distributions on HAR datasets using K-means++. [To view other trials.](#)

Here, however, it can be seen that at $k = 6$ both methods produced more heterogeneous partitionings of the data points, and this is due to the nature of the real-world HAR dataset, where activity records may potentially overlap with one another. Nevertheless, an improvement can be observed in the quantitative data in terms of the stability of the clustering.

The various box plots are shown below (red for standard K-means and blue for K-means++):

4.5 On synthetic datasets

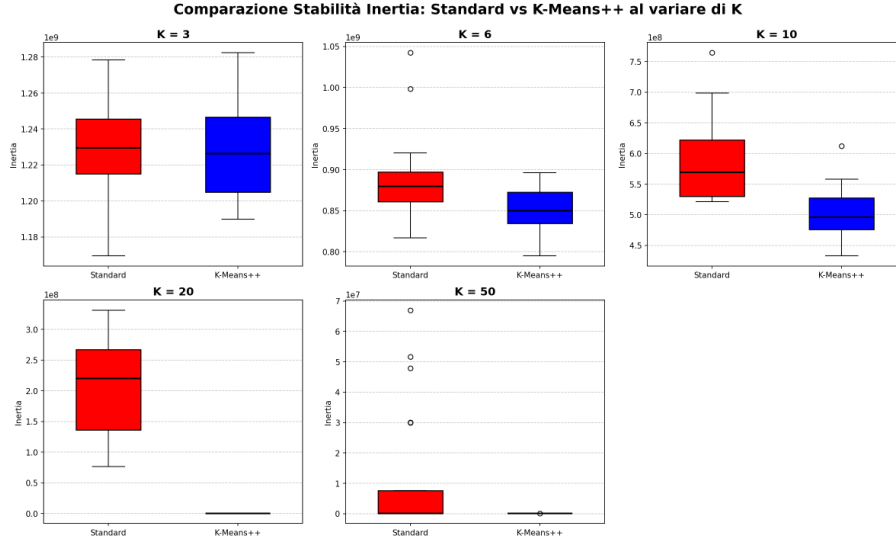


Figure 5: Boxplot on synthetic.

4.6 Su dataset HAR

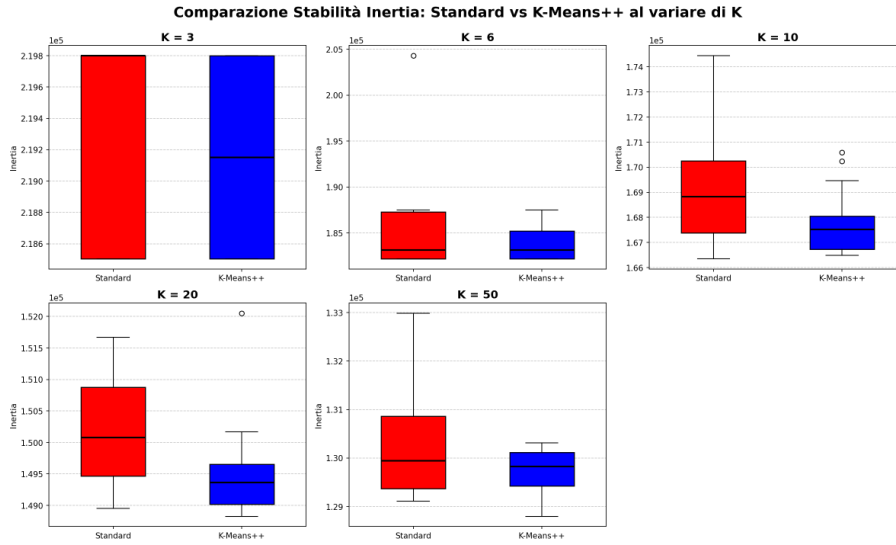


Figure 6: Boxplot on HAR.

When analysing the **Synthetic Dataset (Figure 5)**, the impact of the K-Means++ initialisation is evident. Whilst the Standard algorithm exhibits an extremely wide boxplot along the y-axis, indicating high variance, the boxplot for K-Means++ either collapses into a single horizontal line or exhibits a progressively narrower y-axis span compared to the Standard algorithm.

In the case of the **HAR dataset (Figure 6)**, the greater topological complexity of the data prevents a complete collapse of the variance, but the advantage of K-Means++ remains unchanged. Indeed, a marked compression of the interquartile range can be observed in the blue boxplots compared with the red ones, particularly at $k = 6$ and $k = 50$. This compactness ensures that the results returned by the algorithm are highly predictable and clustered around the median value.

5 Conclusions

Alongside these graphs, quantitative results were also generated:

5.1 Synthetic datasets

K	Algorithm	Inertia	Variability	Time (s)
3	Standard	1.2299×10^9	2.5476×10^7	0.01
	K-Means++	1.2278×10^9	2.4697×10^7	0.01
	<i>Improvements</i>	<i>+0.17%</i>	<i>+3.06%</i>	<i>-27.87%</i>
6	Standard	8.8979×10^8	5.0268×10^7	0.03
	K-Means++	8.5295×10^8	2.5323×10^7	0.02
	<i>Improvements</i>	<i>+4.14%</i>	<i>+49.62%</i>	<i>+12.28%</i>
10	Standard	5.8799×10^8	6.6525×10^7	0.07
	K-Means++	5.0262×10^8	3.9725×10^7	0.05
	<i>Improvements</i>	<i>+14.52%</i>	<i>+40.29%</i>	<i>+26.56%</i>
20	Standard	2.0621×10^8	7.4775×10^7	0.40
	K-Means++	1.1996×10^5	0	0.16
	<i>Improvements</i>	<i>+99.94%</i>	<i>+100.00%</i>	<i>+60.72%</i>
50	Standard	1.1408×10^7	2.0790×10^7	1.37
	K-Means++	1.0987×10^5	2.4865×10^2	2.24
	<i>Improvements</i>	<i>+99.04%</i>	<i>+100.00%</i>	<i>-63.21%</i>

Table 1: Comparison of performance metrics between Standard K-Means and K-Means++ on the **Synthetic** dataset.

5.2 Dataset HAR

K	Algorithm	Inertia	Variability	Time (s)
3	Standard	2.1928×10^5	6.3517×10^2	0.90
	K-Means++	2.1915×10^5	6.4826×10^2	1.02
	<i>Improvements</i>	<i>+0.06%</i>	<i>-2.06%</i>	<i>-13.18%</i>
6	Standard	1.8497×10^5	4.8886×10^3	4.68
	K-Means++	1.8389×10^5	2.1261×10^3	5.34
	<i>Improvements</i>	<i>+0.58%</i>	<i>+56.51%</i>	<i>-14.30%</i>
10	Standard	1.6904×10^5	2.2451×10^3	7.95
	K-Means++	1.6776×10^5	1.2231×10^3	9.90
	<i>Improvements</i>	<i>+0.76%</i>	<i>+45.52%</i>	<i>-24.55%</i>
20	Standard	1.5017×10^5	8.6636×10^2	22.75
	K-Means++	1.4948×10^5	6.9869×10^2	24.90
	<i>Improvements</i>	<i>+0.46%</i>	<i>+19.35%</i>	<i>-9.48%</i>
50	Standard	1.3024×10^5	1.0613×10^3	49.14
	K-Means++	1.2969×10^5	4.8294×10^2	68.92
	<i>Improvements</i>	<i>+0.42%</i>	<i>+54.50%</i>	<i>-40.26%</i>

Table 2: Comparison of performance metrics between Standard K-Means and K-Means++ on the real-world **HAR** dataset.

Numerically, we can see that the K-Means++ algorithm generally yields better results than the standard K-Means algorithm for both types of dataset, both in terms of reduced inertia and inter-trial stability. The underlying cause of this difference lies entirely in the initialisation phase: whilst standard K-Means selects the initial centroids purely at random (thereby running a high risk of placing them too close together or in redundant areas), K-Means++ adopts a probabilistic strategy. By calculating the relative distances between data points, it forces the new centroids to be positioned as far away as possible from those already selected. The trade-off for this ‘intelligent’ and controlled initialisation is, in most cases, a natural increase in the initial computation time.

By analysing the performance differences in detail as the parameter k varies, it is possible to divide the results into three distinct scenarios, highlighting the specific behaviour observed across the two datasets:

- (**k smaller than the actual number of classes**): In this scenario ($k = 3$), the number of required clusters decreases but they are extremely large, making it easy for standard K-means to find a random initial placement. The few iterations required to converge to the final centres do not allow K-Means++ to recoup the time spent on initialisation.
 - *Synthetic dataset*: K-Means++ shows almost no improvement in Inertia and Variance, whilst suffering a significant increase in computation time (-27.87%).
 - *HAR Dataset*: The trend is identical; there is even a slight deterioration in stability (-2.06%) and a slowdown of 13.18%, confirming that for such coarse clusterings, intelligent initialisation is unnecessary.
- (**k coinciding with the natural classes**): in this case ($k = 20$ for the Synthetic dataset, $k = 6$ for the HAR dataset), the algorithm is required to centre the natural clusters of the dataset. It is at this stage that a clear distinction between the two datasets becomes apparent.

- *Synthetic dataset*: by centring the 20 well-separated clusters from the outset, variability and inertia improve significantly (by +99.93% and +100% respectively). The time saved on iterations makes K-means++ significantly faster than the standard version (+60.72%).
- *HAR Dataset*: Given the complex real-world overlap between human activities, the global inertia improves by only a few tenths of a per cent, but structural stability sees a tremendous leap (+56.51%). However, as the centroids struggle to delineate the blurred boundaries between classes, the time advantage evaporates, resulting in a final slowdown of 14.30%.
- (**k greater than the natural classes**): By pushing the value of k to extremes (e.g. $k = 50$), the data is forcibly fragmented. K-Means++ strictly separates the clusters, unlike the random collisions in the standard algorithm, but the computational cost of calculating the relative distances increases.
 - *Synthetic dataset*: The algorithm confirms its geometric infallibility by maintaining variance at zero (a 100% improvement), but the execution time drops dramatically (-63.21%).
 - *HAR Dataset*: The computational limit is equally evident (execution time worsened by 40.26%); despite this, the algorithm yields a more stable variance (54.50%).

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

References

- [1] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms*, pages 1027–1035. Society for Industrial and Applied Mathematics, 2007.